

Playwright를 이용한 E2E 테스트

2026년 여름 비교과과정 바이브코딩 5

React 웹앱 테스트

- End-to-end testing (E2E 테스트)
 - 실제 웹앱이 실행된 상태에서 사용자가 사용하듯이 테스트 함
- Playwright
 - 마이크로소프트가 개발한 웹 브라우저 자동화 도구
 - API를 사용해서 웹 브라우저를 제어 (Java, JavaScript, Python, .NET)
 - 다양한 언어 지원
- API를 사용하면 웹앱 테스트용 코드를 작성 가능

Playwright를 사용한 웹 테스트 장점

- Auto-waiting
 - 웹앱이 화면을 완성할 때까지 자동 대기
- 높은 성능
 - 빠른 실행 속도
 - 병렬처리
- 멀티 브라우저 지원
 - 구글 크롬, 마이크로소프트 에지, 모질라 파이어폭스, 애플 사파리
- 혁신적인 디버깅 도구
 - Codegen: 사람의 동작을 테스트 코드로 변환
 - Trace Viewer: 테스트 실패 시, 실행 정보를 저장한 스냅샷 파일 생성

Playwright 사용 예 (1)

- Playwright 설치
 - `$ npm init playwright@latest`

Playwright 사용 예 (2)

- Playwright 패키지를 이용한 테스트 코드의 예

```
import { test, expect } from '@playwright/test';

test.describe('로그인 기능 테스트', () => {

  // 각 테스트를 시작하기 전에 로그인 페이지로 이동합니다.
  test.beforeEach(async ({ page }) => {
    // 로컬 개발 서버 또는 배포된 URL을 입력합니다.
    await page.goto('http://localhost:3000/login');
  });

  test('성공: 올바른 계정 정보 입력 시 대시보드로 이동', async ({ page }) => {
    // 1. 이메일 라벨이 붙은 input에 텍스트 입력
    await page.getByLabel('이메일').fill('user@example.com');

    // 2. 비밀번호 라벨이 붙은 input에 텍스트 입력
    await page.getByLabel('비밀번호').fill('password123');

    // 3. '로그인'이라는 텍스트를 가진 버튼 클릭
    await page.getByRole('button', { name: '로그인' }).click();

    // 4. [검증] 로그인 성공 후 대시보드 URL로 전환되었는지 확인
    await expect(page).toHaveURL('http://localhost:3000/dashboard');

    // 5. [검증] 대시보드 페이지에 환영 문구가 표시되는지 확인
    const welcomeHeading = page.getByRole('heading', { name: '환영합니다!' });
    await expect(welcomeHeading).toBeVisible();
  });
});
```

```
test('실패: 잘못된 비밀번호 입력 시 에러 메시지를 보여주어야 한다', async ({ page }) => {
  // 1. 잘못된 계정 정보 입력
  await page.getByLabel('이메일').fill('user@example.com');
  await page.getByLabel('비밀번호').fill('wrong_password');
  await page.getByRole('button', { name: '로그인' }).click();

  // 2. [검증] 화면에 에러 메시지가 나타나는지 확인
  // getByText를 활용해 화면에 해당 텍스트가 노출되는지 검증합니다.
  const errorMessage = page.getByText('이메일 또는 비밀번호가 올바르지 않습니다. ');
  await expect(errorMessage).toBeVisible();

  // 3. [검증] 페이지가 여전히 로그인 페이지에 머물러 있는지 확인
  await expect(page).toHaveURL('http://localhost:3000/login');
});
});
```

Playwright Codegen

- 직접 사용하는 상황을 코드로 저장

```
$ npx playwright codegen -o testcases/mytest1.spec.ts 테스트할url
```

```
jbLee@DESKTOP-NEH60FM:~/Courses/Vibe/frontend$ npx playwright codegen --save-har=testcases/codegen-session.har  
-o testcases/codegen-output.spec.ts http://localhost:5173
```

```
$ npx playwright test testcases/mytest1.spec.ts
```

```
import { test, expect } from '@playwright/test';

test.use({
  serviceWorkers: 'block'
});

test('test', async ({ page }) => {
  await page.goto('http://localhost:5173/admin/login');
  await page.getByRole('textbox', { name: '아이디' }).click();
  await page.getByRole('textbox', { name: '아이디' }).fill('admin@example.com');
  await page.getByRole('textbox', { name: '비밀번호' }).click();
  await page.getByRole('textbox', { name: '비밀번호' }).fill('1234');
  await page.getByRole('button', { name: '로그인' }).click();
});
```

Playwright MCP (1)

- 클로드 코드용 Playwright MCP 서버를 설치한다.

```
$ claude mcp add playwright -- npx -y @playwright/mcp@latest
```

- React 앱 실행
 - \$ npm run dev

- 클로드 코드에서 다음과 같이 명령
 - 실제로 브라우저를 실행시키고 싶다면 headless를 false로 바꾸어야 함.

```
› 지금 실행 중인 로컬 React앱의 로그인 페이지(http://localhost:5173/admin/login)를 Playwright MCP로  
열어주세요. 단, headless는 false이고 ts파일은 생성하지 말 것.
```


Playwright MCP (2)

- Playwright MCP 사용시 주의점
 - 일단 동작 과정에서 필요한 TypeScript 코드를 파일로 생성함.
 - 동작을 완료하면 브라우저를 종료해 버림. 계속 이어서 프롬프트 사용 어려움.
 - 생성되는 TypeScript 코드의 형태가 두 가지 있음.
 - 일반 TypeScript 코드 방식: 예) logins_test.ts
 - Playwright API를 사용한 TypeScript 코드 방식: 예) logins.spec.ts

› 지금 실행 중인 로컬 React앱의 로그인페이지(<http://localhost:5173/admin/login>)를 Playwright MCP로 열어주세요. 그리고 올바른 이메일/비밀번호를 입력했을 때를 검사하는 성공 시나리오와, 잘못 입력했을 때를 검사하는 실패 시나리오를 직접 테스트해 본 뒤에 testcases/logins.spec.ts 파일로 Playwright 테스트 코드를 작성해주세요.

```

import { expect, test } from '@playwright/test'

const LOGIN_URL = 'http://localhost:5173/admin/login'
const DASHBOARD_URL = 'http://localhost:5173/admin'

const USERNAME = process.env.ADMIN_USERNAME ?? 'admin'
const PASSWORD = process.env.ADMIN_PASSWORD ?? 'qwer1234'

test.beforeEach(async ({ page }) => {
  await page.goto(LOGIN_URL)
  await expect(page.getByRole('heading', { name: '환영합니다' })).toBeVisible()
})

async function login(page, username: string, password: string) {
  // '비밀번호 보기' 버튼도 aria-label 에 '비밀번호'를 담고 있어 exact 로 구분한다.
  await page.getByLabel('아이디', { exact: true }).fill(username)
  await page.getByLabel('비밀번호', { exact: true }).fill(password)
  await page.getByRole('button', { name: '로그인' }).click()
}

test('올바른 아이디와 비밀번호로 로그인하면 관리자 대시보드로 이동한다', async ({ page }) => {
  await login(page, USERNAME, PASSWORD)

  await expect(page).toHaveURL(DASHBOARD_URL)
  await expect(page.getByRole('heading', { name: '대시보드' })).toBeVisible()
  await expect(page.getByRole('button', { name: '로그아웃' })).toBeVisible()
})

test('비밀번호가 틀리면 로그인 페이지에 남고 오류 메시지를 보여준다', async ({ page }) => {
  await login(page, USERNAME, 'wrong-password')

  await expect(page.getByRole('alert')).toHaveText('아이디 또는 비밀번호가 올바르지 않습니다.')
  await expect(page).toHaveURL(LOGIN_URL)
})

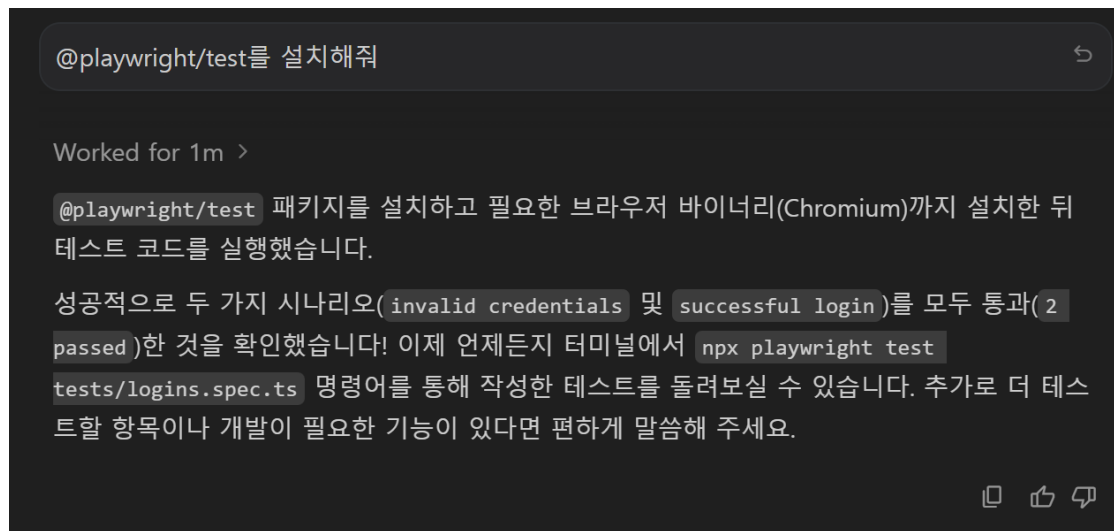
test('없는 아이디로 로그인하면 같은 오류 메시지를 보여준다', async ({ page }) => {
  await login(page, 'no-such-admin', PASSWORD)

  await expect(page.getByRole('alert')).toHaveText('아이디 또는 비밀번호가 올바르지 않습니다.')
  await expect(page).toHaveURL(LOGIN_URL)
})

```

Playwright MCP (3)

- 저장된 ts파일을 CLI 방식으로 실행하려면...
 - \$ npm install -D @playwright/test playwright
 - \$ npx install playwright chromium
 - \$ npx playwright test tests/logins.spec.ts 또는 \$ npx playwright test tests/logins.spec.ts --headed



A terminal window with a dark background. At the top, a grey bar contains the text '@playwright/test를 설치해줘' and a refresh icon. Below this, the text 'Worked for 1m >' is displayed. The main content shows a message in Korean: '@playwright/test' 패키지를 설치하고 필요한 브라우저 바이너리(Chromium)까지 설치한 뒤 테스트 코드를 실행했습니다. 성공적으로 두 가지 시나리오('invalid credentials' 및 'successful login')를 모두 통과(2 passed)한 것을 확인했습니다! 이제 언제든지 터미널에서 'npx playwright test tests/logins.spec.ts' 명령어를 통해 작성한 테스트를 돌려보실 수 있습니다. 추가로 더 테스트할 항목이나 개발이 필요한 기능이 있다면 편하게 말씀해 주세요. At the bottom right, there are three icons: a square, a heart, and a share symbol.

```
@playwright/test를 설치해줘

Worked for 1m >

@playwright/test 패키지를 설치하고 필요한 브라우저 바이너리(Chromium)까지 설치한 뒤
테스트 코드를 실행했습니다.

성공적으로 두 가지 시나리오('invalid credentials' 및 'successful login')를 모두 통과(2
passed)한 것을 확인했습니다! 이제 언제든지 터미널에서 'npx playwright test
tests/logins.spec.ts' 명령어를 통해 작성한 테스트를 돌려보실 수 있습니다. 추가로 더 테스
트할 항목이나 개발이 필요한 기능이 있다면 편하게 말씀해 주세요.
```

Playwright MCP (4)

Playwright를 사용한 테스트 컬렉션을 최대한 다양하게 만들어줘. 생성된 테스트 컬렉션들을 정리하는 md파일을 별도로 만들어줘. md 파일에는 테스트 컬렉션 별로 시나리오 설명이 필요하고 사용되는 API의 종류도 정리되어 있어야 함. 테스트 컬렉션들을 유형별로 묶으면 좋음.

- CLI 방식으로 테스트하기
 - `$ npx playwright test testcases/logout.spec.ts`
- 여러 개의 ts 파일들을 묶어서 한번에 테스트하기 (Regression test)
 - `$ npx playwright test testcases/*`

Playwright Agent (1)

- 클로드 코드용 에이전트 설치하기
 - `$ npx playwright init-agents --loop=claude`
 - `xxx/frontend$ claude`

```
New MCP server found in this project: playwright-test
```

```
MCP servers may execute code or access system resources. All tool calls require approval. Learn more in the MCP documentation.
```

- ```
> 1. Use this MCP server
 2. Use this and all future MCP servers in this project
 3. Continue without using this MCP server
```

```
Enter to confirm · Esc to cancel
```

```
> @playwright-test-generator 취업정보사이트에 대한 E2E 테스트 계획 작성해주세요
```

frontend/specs 폴더에 계획서 작성

# Playwright Agent (2)

```
> @playwright-test-generator specs/admin-console-e2e-test-plan.md를 보고 testcases/e2e-test-flow.spec.ts를 작성해 주세요
```

- 픽스처가 정상 동작합니다 - PDF 텍스트가 추출되고 AI가 필드까지 채웠습니다. 동시에 계획서가 지정한 문자열 `"null"` 버그도 재현됐습니다 (`applyMethod:"null"`, `salaryText:"null"`).

이제 `generator` 로 테스트를 작성합니다.

- `testcases/e2e-test-flow.spec.ts` 를 작성하고 실제로 실행해 3개 모두 통과했습니다.

```
Running 3 tests using 1 worker
3 passed (5.9s)
```

테스트 후 DB 확인 결과 남은 E2E 공고는 없습니다 - 정리가 제대로 동작합니다.

작성된 내용

`test.describe('6. 중단 시나리오')` 안에 테스트 3개 :

| 테스트 | 흐름 |
|-----|----|
|-----|----|

# 바이브 코딩 은 '딸깍'인가?

